DEPLOYMENT PLAN

UMHackathon 2026

umhackathon@um.edu.my

Email: umhackathon@um.edu.my

Table of Contents

Email: umhackathon@um.edu.my

1 Overview

Project Name:	AdKedai
Project Start:	2/5/2026
Current Date:	3/5/2026

Task	Project Lead	Start Date	End Date	Days	Status	Comments/Progress
Setup frontend deployment on Vercel	Lim Tze Jiun	2/5/2026	2/5/2026	1	Completed	Connected GitHub repo to Vercel for auto deployment
Configure backend deployment on AWS EC2	Er Qing Yap	2/5/2026	2/5/2026	1	Completed	FastAPI server deployed using Uvicorn
Integrate Firestore database	Lim Chun Rong	2/5/2026	2/5/2026	1	Completed	Database connected for storing inventory and results
Configure environment variables (API keys)	Wong Yee Xiong	2/5/2026	2/5/2026	1	Completed	GLM API keys stored securely in environment variables
Implement CI/CD pipeline via Vercel	Lim Tze Jiun	2/5/2026	2/5/2026	1	Completed	Auto deployment triggered on Git push
Deploy initial production version	Chong Yun Enn	2/5/2026	2/5/2026	1	Completed	First stable version deployed successfully
Test deployment (end-to-end flow)	All Team Members	3/5/2026	3/5/2026	1	Completed	Verified upload → recommendation → content generation
Fix deployment issues & bugs	Wong Yee Xiong	3/5/2026	3/5/2026	1	Completed	Resolved API response delay issue
Validate AI output in production	Lim Chun Rong	3/5/2026	3/5/2026	1	Completed	Ensured GLM outputs are consistent and accurate
Final deployment review & readiness check	Lim Chun Rong	3/5/2026	3/5/2026	1	Completed	System ready for demo and final presentation

Figure 1 Deployment Plan Table

This document presents the Deployment Plan and Production Architecture of the AdPilot system. It outlines the architecture design, technical stack, scalability strategy, CI/CD pipeline, and security integration of the system to demonstrate production readiness and technical feasibility.

AdPilot is designed as a cloud-based AI decision intelligence platform that transforms inventory data into advertising strategies and content using a structured multi-stage pipeline powered by GLM. The system integrates frontend, backend, AI services, and database components to ensure efficient data processing and seamless user interaction.

The deployment architecture emphasizes:

- **Scalability** - supporting multiple users and large inventory datasets through asynchronous processing, chunking mechanisms, and cloud-based infrastructure
- **Automation** - enabling continuous deployment via Vercel with minimal manual intervention
- **Reliability** - ensuring stable system performance through validation, logging, and controlled deployment behavior
- **Security** - protecting user data and system access through authentication, credential management, and secure communication protocols

This document demonstrates how the system is designed not only as a prototype but as a production-ready solution, with considerations for performance, maintainability, and real-world deployment.

Email: umhackathon@um.edu.my

2 System Architecture, Technical Stack & Scalability

2.1 System Architecture Diagram & Explanation

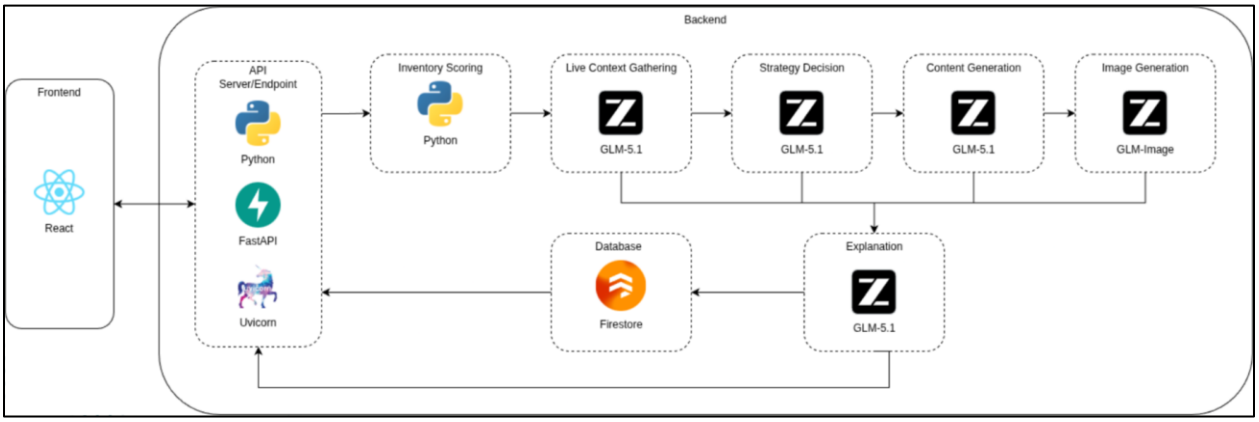

Figure 2 System Architecture Diagram

2.1.1 Frontend Layer

React (Frontend)

The frontend is developed using **React**, which provides the user interface for interacting with the system.

It is responsible for:

- Accepting user inputs (CSV and multi-format uploads)
- Displaying generated recommendations and strategies
- Allowing users to select preferred strategies
- Presenting generated ad content and images
- Handling export and user interactions

The frontend communicates with the backend through REST API calls.

2.1.2 Backend Layer

API Server / Endpoint

Acts as the entry point of the system.

Responsibilities:

- Receive requests from the frontend

- Validate and preprocess uploaded files
- Convert multi-format inputs into structured CSV format
- Route data to appropriate processing modules

Inventory Scoring

Processes and analyzes the uploaded inventory data.

Functions:

- Evaluate stock levels, pricing, and categories
- Identify priority, overstock, or new products
- Prepare structured input for AI processing

Live Context Gathering (GLM-5.1)

Enrich inventory data with real-world context.

Includes:

- Market trends
- Platform trends
- Seasonal events

This improves the relevance of AI-generated decisions.

Strategy Decision (GLM-5.1)

Generates advertising strategies based on processed data.

Outputs:

- Platform selection
- Target audience
- Pricing strategy
- Posting schedule

Multiple ranked strategies are generated for user selection.

Email: umhackathon@um.edu.my

Content Generation (GLM-5.1)

Produces advertising content based on selected strategy.

Includes:

- Headlines
- Captions
- Hashtags
- Campaign messaging

All outputs are aligned with product data and strategy.

Image Generation (GLM-Image)

Generates visual assets for advertising.

Produces:

- Product-based promotional images
- Platform-ready visual content

Explanation (GLM-5.1)

Provides reasoning behind AI-generated recommendations.

Purpose:

- Improve transparency
- Help users understand decision logic
- Increase trust in the system

Database (Firebase Firestore)

Stores all system data and user-related information.

Includes:

- User accounts and authentication data
- Uploaded inventory records
- Generated recommendations and content
- Session history and cached results

This supports faster retrieval and reduces repeated AI processing.

Email: umhackathon@um.edu.my

2.2 Technical Stack Table

Layer	Technology	Purpose
Frontend	React (Vercel)	Provides user interface and handles user interactions
Backend	FastAPI (Python)	Handles API requests, data processing, and system logic
Server	Uvicorn	Runs FastAPI application with high-performance ASGI server
AI Engine	GLM-5.1	Generates advertising strategies and content
Image Generation	GLM-Image	Produces AI-generated promotional visuals
Database	Firebase Firestore	Stores user data, inventory, and generated outputs
Deployment	AWS EC2	Hosts backend services and manages server infrastructure

2.3 Scalability Flow / Strategy

The system is designed to handle increasing user demand and large-scale data processing through the following mechanisms:

- **Asynchronous Backend Processing**
  FastAPI and Uvicorn support non-blocking request handling, allowing multiple users to interact with the system simultaneously.
- **CSV Chunking Mechanism**
  Large inventory files are split into smaller batches (100–150 rows), preventing token overflow and improving processing efficiency.
- **Caching with Firestore**
  Previously processed data is stored and reused, reducing redundant AI calls and improving response time.
- **External AI Processing (GLM API)**
  Heavy computation is offloaded to GLM services, reducing backend resource usage.

Email: umhackathon@um.edu.my

- **Cloud-Based Deployment (AWS & Vercel)**
  The system can scale horizontally to support higher traffic and user demand.

3 CI/CD Pipeline & Automation

3.1 CI/CD Pipeline Overview

The system implements an automated **Continuous Deployment (CD)** pipeline using **Vercel**, enabling seamless frontend deployment directly from the code repository.

The pipeline ensures that every code update is automatically built, tested at the platform level, and deployed without manual intervention.

3.2 CI/CD Pipeline Flow

Step	Process	Description
1	Code Push	Developer pushes code to the repository (e.g., GitHub)
2	Trigger Detection	Vercel detects new commit via Git integration
3	Dependency Installation	Required packages are installed automatically
4	Build Process	Frontend application is compiled and built
5	Build Validation	System verifies build success or failure
6	Deployment	Successful build is deployed to production or preview
7	Monitoring	Deployment logs are generated for tracking and debugging

3.3 Automation Mechanism

Automation in this project refers to the execution of deployment processes without manual intervention. The system leverages **Vercel's built-in automation** to handle:

- Automatic build and deployment triggered by Git commits
- Continuous integration between repository and hosting platform
- Real-time deployment tracking through logs and commit history

This ensures a consistent and reliable deployment workflow across all team members.

3.4 Deployment Behavior

The deployment pipeline follows a branch-based deployment strategy:

- **Production Deployment**

  Triggered automatically when code is pushed to the main branch

  - Deploys stable version of the application
- **Preview Deployment**

  Triggered for feature branches or pull requests

  - Allows testing before merging into production
- **Failure Handling**

  If the build process fails:

  - Deployment is automatically blocked
  - Error logs are generated for debugging

3.5 Benefits of CI/CD Automation

The automated pipeline provides several advantages:

- **Faster Release Cycle**

  Eliminates manual deployment steps and accelerates updates
- **Consistency**

  Ensures all deployments follow the same standardized process
- **Error Reduction**

  Minimizes human errors during deployment
- **Traceability**

  Each deployment is linked to a specific commit and can be tracked easily
- **Rollback Capability**

  Previous stable versions can be redeployed quickly if needed

Email: umhackathon@um.edu.my

4 Security Integration & Credential Management

The system implements secure integration practices to protect user data, API access, and system resources. Security measures focus on authentication, credential storage, data protection, and controlled access to external services.

Component	Security Measure	Description
Authentication	Firebase Authentication	Manages user login and access control securely using token-based authentication
API Keys	Environment Variables	Sensitive API keys (e.g., GLM API) are stored in environment variables and never exposed in frontend code
Data Transmission	HTTPS Protocol	All client-server communication is encrypted using HTTPS to prevent data interception
Database Access	Role-Based Access Control	Firestore rules restrict access to authorized users only
Input Validation	Backend Validation	Uploaded files are validated before processing to prevent malformed or malicious input
AI Output Control	Schema Validation	JSON schema validation ensures AI outputs follow expected structure and prevents invalid data usage
Deployment Security	Vercel & AWS Security	Secure deployment environments with controlled access and managed infrastructure
Logging & Monitoring	Deployment Logs	System logs track activities and errors for debugging and auditing purposes

Email: umhackathon@um.edu.my